

Lab 3 – Architectural Justification

Problem Statement: Smart Complaint Management System

Section D | PES University

Architecture Selection

“We chose a Microservices Architecture for the Smart Complaint Management System.”

The Citizen/Staff Web & Mobile App talks only to an **API Gateway**, which routes requests to six independently deployable services: **Identity & User** (FR-010, UC-09), **Complaint** (FR-001, FR-003, FR-005, FR-012, UC-01 to UC-05), **Categorisation** (FR-002, UC-07), **Routing & SLA** (FR-004, FR-006, FR-009, UC-06, UC-10), **Notification** (FR-007) and **Analytics** (FR-011, CSAT from FR-008, UC-08). Each service owns its data store and exposes a defined interface. The Complaint Service orchestrates a submission by calling ICategorise, IRouting and INotify, and publishes status events that Analytics consumes for the dashboard. External email/SMS and geocoding providers sit behind their own services.

Reason 1 – Very different workloads in one system

NLP categorisation runs model inference on every new complaint (UC-01 step 3, UC-07) and is CPU- and memory-heavy, the dashboard is read-heavy and refreshes every 60 s for every supervisor (UC-08), while user management and SLA configuration are rare admin actions. With microservices we can scale only the Complaint and Categorisation services horizontally during submission peaks (e.g. after a storm or water outage) to meet NFR-001 (500 concurrent users, API p99 under 500 ms), and update the NLP model by redeploying Categorisation alone instead of the entire application.

Reason 2 – Fault isolation and graceful degradation

The NLP model and the third-party email/SMS gateway are the parts most likely to be slow or fail. UC-07 already expects manual category selection when confidence is below 70% (alternate flow 3a); because Categorisation is a separate service with a timeout, the same fallback applies if it is down, so citizens can still submit. Notifications go through an asynchronous queue, so a slow SMS provider never blocks submission, resolution or escalation (FR-007 allows delivery within 60 s). This containment supports NFR-003 (99.9% uptime), and SLA rules changed by the Admin (UC-10) take effect in Routing & SLA without redeploying complaint handling.

Security Advantage – Citizen data isolated with least privilege

Citizen contact details live only in the Identity & User database, and complaint text, GPS locations, photos and evidence (UC-05) only in the Complaint database, both AES-256 encrypted at rest (NFR-002). Other services receive only what they need: Categorisation gets the description text and stores nothing, Routing & SLA gets the complaint ID, category and zone, never photos or phone numbers. The API Gateway is the single public entry point: it terminates TLS 1.3, validates JWT tokens, enforces the four RBAC roles (FR-010) and rate-limits clients, while internal services are not reachable from the internet. Only the Complaint Service can write the append-only audit log (FR-012), so a compromise of the read-only Analytics service cannot alter the complaint history.

Performance Benefit – Fast submission, cheap dashboards

The latency-critical paths are submitting and tracking complaints (UC-01, UC-02). Categorisation keeps the model loaded in memory, so a suggestion is a single stateless call, and notifications are queued, so the citizen gets the ticket ID as soon as it is saved and routed. Analytics keeps pre-aggregated KPIs in its own store, built from the event stream, so dashboard refreshes never run heavy aggregate queries on the live complaint database. In a client-server or monolithic design, NLP inference, dashboard queries and complaint writes would compete for the same server and database, and a load spike in one would slow all of them.

Alternatives Considered and Limitations

Layered (monolith) is simpler to build, test and deploy, and a single database makes status updates, ratings and KPIs trivially consistent; it would suit a single-ward pilot, but it must be scaled as a whole and a hung SMS or model call can affect every feature. **Client-Server** offers centralised control and easy data consistency, but one server is a scalability bottleneck during complaint spikes and a single point of failure. **Pure event-driven** would suit notifications and analytics (we use events there) but makes request/response flows such as tracking status (UC-02) harder to reason about and debug. **Limitations of our choice:** higher operational complexity (deploying and monitoring six services plus a message queue), extra network hops, and eventual consistency: the dashboard can lag the live complaint state by a few seconds, which is acceptable given its 60 s refresh. A submission spans Complaint and Routing & SLA, so it needs a compensating step: if routing fails after the ticket is saved, the complaint stays "New" in an unassigned queue, routing is retried, and the supervisor is alerted, instead of the citizen losing the complaint.